课程介绍

- 有监督学习
    1. 分类
        - KNN
        - 逻辑回归
        - 决策树
    2. 回归
        - 线性回归
        - 岭回归
        - lasso回归
    3. 集成学习
- 无监督学习

近邻算法

KNN

- K--K个
- N--nearest　　最近
- N--neighbor　邻居

K个最近的邻居决定我们的类别

什么是K-近邻算法

- 根据你的“邻居”来推断出你的类别

K-近邻算法(KNN)概念

k个最相似(即特征空间中最邻近)的样本中的大多数属于某一个类别

KNN算法流程

1. 计算距离
2. 距离排序
3. 选取最近的K个点
4. 统计K个邻居类别出现最多的那个类别
5. 预测我的所属类别

K值的选择（奇数）

邻居计算（欧氏距离）

k近邻算法api初步使用

复习流程：

- 1.获取数据集
- 2.数据基本处理
- 3.特征工程
- 4.建立模型
    - KNN
- 5.模型评估

Scikit-learn工具介绍

- Python语言的机器学习工具
- Scikit-learn包括许多知名的机器学习算法的实现
- Scikit-learn文档完善，容易上手，丰富的API
- 目前稳定版本0.19.1

K-近邻算法API

`sklearn.neighbors.KNeighborsClassifier(n_neighbors=5)`

- n_neighbors：int,可选（默认= 5），k_neighbors查询默认使用的邻居数
- 建立模型
  1. 实例化api
     - `estimator = KNeighborsClassifier(n_neighbors=2)`
  2. 训练模型
     - `estimator.fit(x, y)`
  3. 模型预测
     - `` `estimator.predict() ``

距离度量

1. **欧式距离(Euclidean Distance)：**
   - 两点之间的直线距离
2. **曼哈顿距离(Manhattan Distance)：**
   - 每个维度差值绝对值和
3. **切比雪夫距离 (Chebyshev Distance)：**
   - 每个维度差绝对值最大值
4. **闵可夫斯基距离(Minkowski Distance)：**
   - 其中p是一个变参数：
     - 当p=1时，就是曼哈顿距离；
     - 当p=2时，就是欧氏距离；
     - 当p→∞时，就是切比雪夫距离。
     - 根据p的不同，闵氏距离可以表示某一类/种的距离。

闵氏距离的缺点：

(1)将各个分量的量纲(scale)，也就是“单位”相同的看待了；

(2)未考虑各个分量的分布（期望，方差等）可能是不同的。

5. **标准化欧氏距离 (Standardized EuclideanDistance)：**

标准化欧氏距离公式：

$$d_{12} = \sqrt{\sum_{k=1}^{n} \left(\frac{x_{1k} - x_{2k}}{s_k}\right)^2}$$

6. **余弦距离(Cosine Distance)**

$$\cos\theta = \frac{x_1 x_2 + y_1 y_2}{\sqrt{{x_1}^2 + {y_1}^2}\sqrt{{x_2}^2 + {y_2}^2}}$$

- 取值[-1,1] 取值越大: 关系越紧密

7. 汉明距离(Hamming Distance)【了解】:
   - 两个等长字符串s1与s2的汉明距离为：将其中一个变为另外一个所需要作的最小字符替换次数。
   - 汉明重量:
     - 1的个数
   - 应用:
     - 汉明重量分析在包括信息论、编码理论、密码学等领域都有应用
8. 杰卡德距离(Jaccard Distance)【了解】:

$$J_\delta(A,B)=1-J(A,B)=\frac{|A\cup B|-|A\cap B|}{|A\cup B|}$$

9. 马氏距离(Mahalanobis Distance)【了解】:

$$当m=1时,d^2(x,G)=\frac{(x-\mu)'(x-\mu)}{\sigma^2}=\frac{(x-\mu)^2}{\sigma^2}$$

   - 特征:
     1. **量纲无关**
     2. **马氏距离的计算是建立在总体样本的基础上的**
     3. 计算马氏距离过程中， **要求总体样本数大于样本的维数**

k值的选择

- **K值过小：**容易受到异常点的影响
  - **整体模型变得复杂，容易发生过拟合**
- **k值过大：**受到样本均衡的问题
  - **与输入实例较远（不相似的）训练实例也会对预测器作用，使预测发生错误，且K值的增大就意味着整体的模型变得简单。**
- 近似误差:
  - 训练集误差　　小---过拟合
- 估计误差:
  - 测试集误差　　小---模型接近最佳模型

kd树

最快找到邻居的数据结构

- 最快找到邻居
- 数据量大，计算量大
- 提高KNN搜索的效率， 可以考虑使用特殊的结构存储训练数据，以减小计算距离的次数。

kd树简介

- **如果A和B距离很远，B和C距离很近，那么A和C的距离也很远**

构造方法

- 找到根节点
- 通过递归不断地切割
- 直到没有实例

（1）选择向量的哪一维度进行划分：

- 选择离散程度比较大的那个维度

（2）如何划分数据：

- 选择中位数，构造平衡的树

最近领域的搜索

候选超球：当前点为圆心最近邻居为半径绘制一个圆

回溯队列：样本需对比的所有点

案例：鸢尾花种类预测--数据集介绍

scikit-learn中数据集介绍

scikit-learn数据集API介绍

```
iris.data   # 特征数据
iris.target # 目标数据
iris.DESCR  # 描述
iris.feature_name   # 特证名
```

数据集的划分

- 训练集　构建模型　80 - 70
- 测试机　评估模型　20 - 30

数据集划分api

`x_train,x_test,y_train,y_test = sklearn.model_selection.train_test_split(x,y,test_size=0.2 ,random_state =22)`

- 参数：
  - x 数据集的特征值
  - y 数据集的标签值
  - test_size 测试集的大小，一般为float
  - random_state 随机数种子,不同的种子会造成不同的随机采样结果。相同的种子采样结果相同。
- return
  - x_train, x_test, y_train, y_test

特征工程-特征预处理

- 通过 **一些转换函数** 将特征数据 **转换成更加适合算法模型** 的特征数据过程(数据无量纲化)

为什么我们要进行归一化/标准化？

- 特征的 **单位或者大小相差较大，或者某特征的方差相比其他的特征要大出几个数量级**， **容易影响（支配）目标结果**，使得一些算法无法学习到其它的特征
- **无量纲化**，**使不同规格的数据转换到同一规格

包含内容(数值型数据的无量纲化)

- 归一化
- 标准化

归一化

- 将原始数据映射到[0,1]
- api : `sklearn.preprocessing.MinMaxScaler(feature_range=())`
- 返回: ndarray
- 总结:
  - 归一化容易受到异常点影响,
  - 应用场景:只适用数据精确,小数据场景

标准化

定义:

- 将数据转化成均值为0 标准差为1 范围内
- 受异常点影响小

api:`sklearn.preprocessing.StandardScaler( )`

返回值：转换后的形状相同的array

- 总结:
  - 比较适合大数据场景

1.8 案例：鸢尾花种类预测—流程实现

sklearn.neighbors.KNeighborsClassifier(n_neighbors=5,algorithm='auto')

- n_neighbors:
  - int,可选（默认= 5），k_neighbors查询默认使用的邻居数
- `algorithm：{‘auto’，‘ball_tree’，‘kd_tree’，‘brute’}`
  - 快速k近邻搜索算法，默认参数为auto，可以理解为算法自己决定合适的搜索算法。除此之外，用户也可以自己指定搜索算法ball_tree、kd_tree、brute方法进行搜索，
  - brute是蛮力搜索，也就是线性扫描，当训练集很大时，计算非常耗时。
  - kd_tree，构造kd树存储数据以便对其进行快速检索的树形数据结构，kd树也就是数据结构中的二叉树。以中值切分构造的树，每个结点是一个超矩形，在维数小于20时效率高。
  - ball tree是为了克服kd树高维失效而发明的，其构造过程是以质心C和半径r分割样本空间，每个节点是一个超球体。

2.2 步骤分析

1. 获取数据
2. 数据基本处理

   2.1 数据切割
3. 特征工程

   3.1 数据标准化
4. 建立模型

   4.1 KNN

5. 模型评估

k近邻算法总结

- 优点:
  1. 简单有效
  2. 重新训练的代价低
  3. 适合 **类域交叉样本**
  4. **适合大样本自动分类**
- 缺点:
  1. 懒惰学习
  2. 类别评分不是规格化
  3. 输出可解释性不强
  4. 对不均衡的样本不擅长
  5. 计算量大